

EEE 304 (July 2024)

Digital Electronics Laboratory

Final Project Report

Section: B2 Group: 03

Development of an LED Display Billboard Using an CPLD Board

Course Instructors:

Nafis Sadik, Lecturer

Rafichha Yasmin, Part-Time Lecturer

Signature of Instructor: _____

Academic Honesty Statement:

IMPORTANT! Please carefully read and sign the Academic Honesty Statement, below. Type the student ID and name, and put your signature. You will not receive credit for this project experiment unless this statement is signed in the presence of your lab instructor.

"In signing this statement, We hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completing this project. We understand that if we fail to honor this agreement, We will each receive a score of ZERO for this project and be subject to failure of this course."

Signature: _____

Full Name: Md. Kaidul Islam

Student ID: 2006111

Signature: _____

Full Name: Sadaf Shafin Chowdhury

Student ID: 2006113

Signature: _____

Full Name: Shohan Ahmed Rifat

Student ID: 2006116

Signature: _____

Full Name: Fabliha Labiba

Student ID: 2006119

Table of Contents

1	Abstract.....	1
2	Introduction.....	1
	2.1 Background.....	1
	2.2 Objective.....	1
	2.3 Scope.....	1
3	Design	1
	3.1 Problem Formulation (PO(b)).....	1
	3.1.1 Identification of Scope.....	1
	3.1.2 Formulation of the Problem	2
	3.2 Key Challenges	2
	3.3 Analysis	2
	3.3.1 Hardware Components.....	2
	3.4 System Architecture.....	4
	3.5 Verilog Design.....	4
	3.6 Performance Metrics.....	4
	3.7 Design Method (PO(a)).....	4
	3. 8 Simulation Model	5
	3.8.1 Simulation Tools and Process.....	5
	3.9 Post-Simulation Adjustments.....	6
4	Implementation	6
	4.1 Hardware Setup.....	6
	4.2 Software Development.....	7
	4.3 Testing	8
	4.4 Results.....	8
	4.5 Github Link.....	8
	4.6 Google Drive Link	8
5	Design Analysis and Evaluation	9
	5.1 Novelty.....	9
	5.2 Performance	9
	5.3 Resource Utilization	9
	5.4 Reliability.....	9
	5.5 Limitations	9
	5.6 Results and Analysis.....	9

6	Reflection on Individual and Team work (PO(i)).....	10
6.1	Individual Contributions	10
6.2	Team Dynamics	10
6.3	Lessons Learned	10
6.4	Log Book of Project Implementation	11
7	Executive Summary	13
7.1	Executive Summary	13
7.2	User Manual.....	13
7.2.1	Introduction.....	13
7.2.2	System Components.....	13
7.2.3	Operating Instructions.....	13
7.2.4	Troubleshooting	14
8	Project Management and Cost Analysis (PO(k)).....	14
8.1	Project Management	14
8.2	Cost Analysis	14
8.3	Budget Justification	14
9	Future Work (PO(l))	15
9.1	Enhancements	15
9.2	Additional Features.....	15
9.3	Scalability	15
10	References	15
11	Appendix	16

1 Abstract

This report presents the development of a digital signage system utilizing an 8x8 dot matrix display controlled by an LP-2900-CPLD development board. The project encompasses the design, implementation, and evaluation of the system, which is capable of displaying various messages and animations based on user inputs. The digital signage leverages Verilog programming to manage display patterns, scrolling text, and interactive features through switch inputs. The system demonstrates efficient use of programmable logic to achieve dynamic visual outputs, offering a cost-effective and scalable solution for digital advertising and informational displays.

2 Introduction

2.1 Background

Digital signage has become an integral component in modern advertising, information dissemination, and user interaction across various environments such as retail stores, public transportation, and corporate offices. It offers dynamic content display capabilities, allowing for real-time updates and interactive features that enhance user engagement.

2.2 Objective

The primary objective of this project is to design and implement a digital signage system using an 8x8 dot matrix LED display controlled by an LP-2900-CPLD development board. The system aims to provide customizable display patterns, scrolling text, and interactive animations based on user inputs via switches, demonstrating the capabilities of programmable logic devices in creating versatile display solutions.

2.3 Scope

The project encompasses the hardware setup involving the CPLD and LED matrix, the development of Verilog code to control display functionalities, and the evaluation of the system's performance. Additionally, the project includes considerations for project management, cost analysis, and potential future enhancements.

3 Design

3.1 Problem Formulation (PO(b))

3.1.1 Identification of Scope

Complexity as an Engineering Problem

This problem qualifies as an engineering problem due to:

- **Interdisciplinary Knowledge Requirements:** It involves knowledge of digital electronics, programmable logic, hardware-software interfacing, and timing control.
- **Innovative Approach:** Designing efficient algorithms in Verilog to handle real-time dynamic visualizations.
- **Real-World Constraints:** Addressing challenges such as power limitations, timing synchronization, and user interface simplicity within a low-cost framework.

Alternative Solutions

- **Microcontroller-Based Systems:** These could provide higher flexibility and easier programmability, but might not match the cost-effectiveness and power efficiency of a CPLD-based design.
- **Pre-Configured Display Modules:** Off-the-shelf modules offer ready-made solutions but lack customization and are generally less resource-efficient.
- **High-Resolution Displays:** Utilizing larger or higher-resolution LED matrices could provide better visual output but would increase cost and complexity.

By choosing a CPLD-based approach, this project aims to optimize the trade-offs between cost, scalability, and performance, offering a tailored solution for small-scale digital signage applications.

3.1.2 Formulation of the Problem

The primary problem this project seeks to address is the design and implementation of a compact and cost-effective digital signage system capable of displaying dynamic visual content such as patterns, scrolling text, and animations. This is particularly challenging due to the constraints imposed by the hardware (an 8x8 dot matrix LED display and LP-2900-CPLD development board) and the requirement to achieve high functionality using programmable logic.

3.2 Key Challenges

1. **Hardware Constraints:** The 8x8 matrix has a limited resolution, requiring efficient design to ensure visually comprehensible patterns and animations.
2. **Programmable Logic Utilization:** Maximizing the functionality of the LP-2900 CPLD while ensuring resource-efficient implementation of features such as scrolling text and animations.
3. **Interactive Features:** Designing a robust and intuitive user interface using simple switches for mode control and animation triggers.
4. **Synchronization:** Achieving precise timing for animations and text scrolling to avoid flickering or glitches in the display.
5. **Scalability and Cost:** Creating a system that is both scalable for larger implementations and affordable for small-scale deployment

3.3 Analysis

3.3.1 Hardware Components

- **LP2900-CPLD Development Board:** Serves as the core programmable logic device managing the display operations.
- **8x8 Dot Matrix LED Display:** The visual output component capable of displaying various patterns and text.

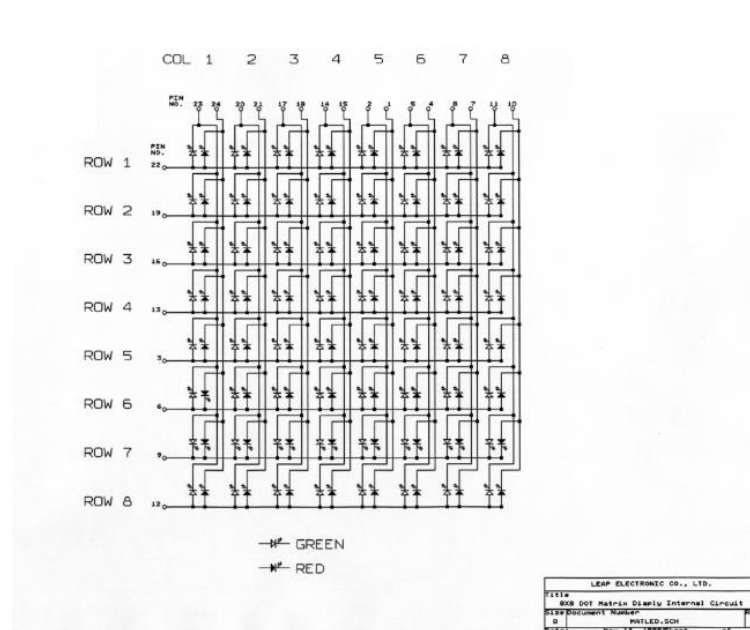


Figure: Internal Connection of 8x8 Dot Matrix Module

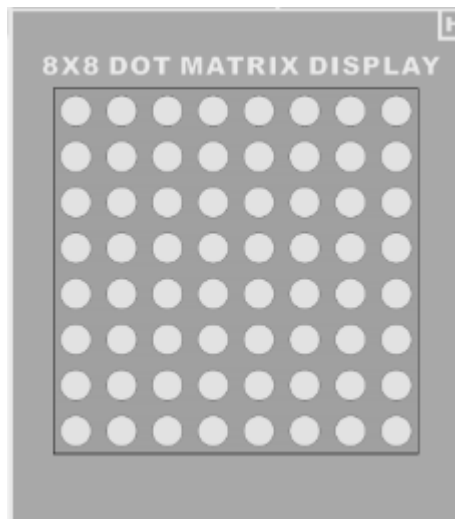


Figure: 8x8 Dot Matrix Display

- **Switches (SW1 to SW8):** User input interfaces to control different display modes and animations.

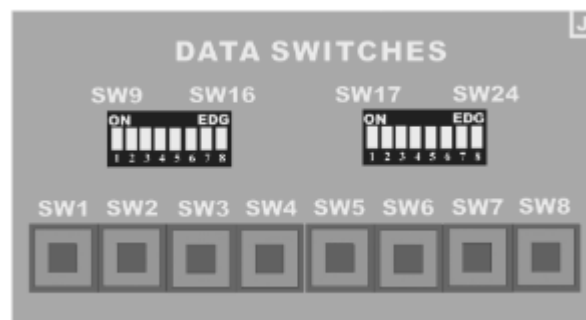


Figure: Data Switches

Code	SW1	SW2	SW3	SW4	SW5	SW6	SW7	SW8
Device	Push Button							
Pin	Pin 47	Pin 48	Pin 49	Pin 51	Pin 59	Pin 60	Pin 62	Pin 63

- **Clock Signal:** Provides the necessary timing signals for synchronous operations within the CPLD.
- **Power Supply:** Ensures stable power delivery to all components.

3.4 System Architecture

The system architecture is centered around the CPLD, which interfaces with the LED matrix and user switches. The CPLD handles multiple functionalities including:

- **Row and Column Control:** Manages the activation of specific rows and columns to display desired patterns.
- **Scrolling Text:** Implements horizontal and vertical scrolling of text messages across the matrix.
- **Animation Frames:** Displays simple animations, such as a walking person, by cycling through predefined frames.
- **User Interaction:** Responds to switch inputs to change display modes, control scrolling, and trigger animations.

3.5 Verilog Design

The Verilog module- ‘Open’ is designed to manage the display logic. Key features include:

- **Display Memory:** Stores binary representations of characters and animation frames.
- **Clock Management:** Generates various clock signals for different timing requirements (e.g., scrolling speed, animation frame rate).
- **Control Logic:** Determines which display mode to activate based on switch inputs.
- **Scrolling Mechanism:** Implements a counter-based approach to scroll text across the display.
- **Animation Control:** Cycles through animation frames to create movement effects.

3.6 Performance Metrics

- **Responsiveness:** Minimal delay in response to user inputs, ensuring a seamless interactive experience.
- **Flicker-Free Output:** Precise timing eliminates display glitches during pattern changes or animations.
- **Resource Utilization:** Efficient use of CPLD resources ensures scalability for larger displays or additional features.

3.7 Design Method (PO(a))

The design process for the digital signage system followed a systematic approach, integrating hardware setup, software development, and iterative testing to ensure functionality and efficiency.

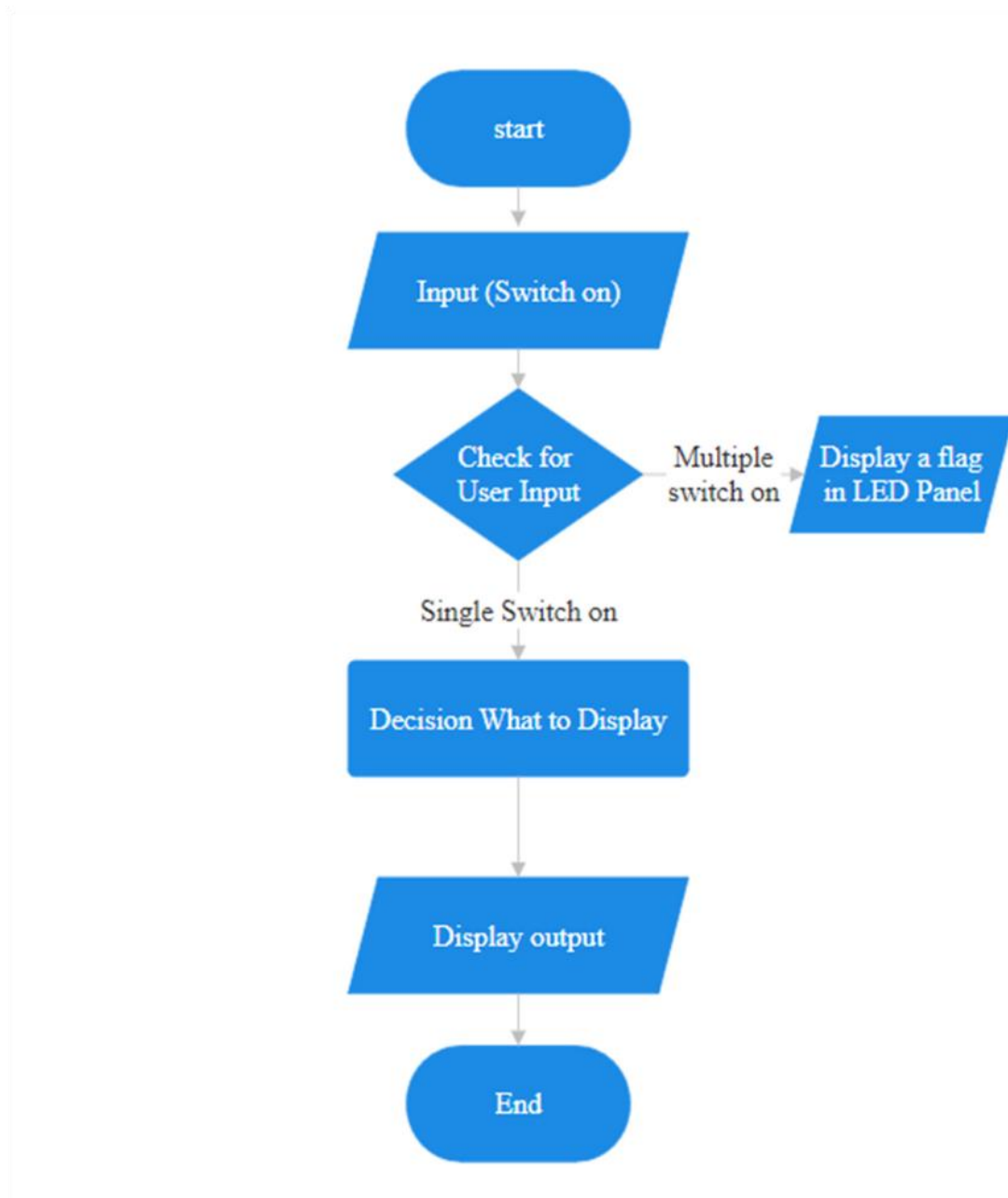


Figure 1 Flow Chart of Design Process

3. 8 Simulation Model

The **Quartus Altera** software was instrumental in simulating and verifying the functionality of the digital signage system before hardware implementation.

3.8.1 Simulation Tools and Process

1. Simulation in Quartus Altera:

- The software provided a comprehensive environment for simulating the Verilog design, observing waveforms, and identifying potential issues.
- A detailed testbench was used to generate input signals (e.g., clock pulses, switch inputs).

2. Simulation Workflow:

- **Testbench Initialization:** Defined all inputs, including clock signals and switch states, and initialized the display with a default pattern.
- **Module Verification:** Tested each module individually for proper functionality under various scenarios.
- **Integrated Testing:** Simulated the entire system to ensure coordinated operation of all modules.

3. Output Validation:

- Observed row and column activations, ensuring correct timing and synchronization.
- Simulated **LED patterns** to verify correct representation of text and animations.
- Tested **mode switching** and other interactive features, ensuring real-time responsiveness.

3.9 Post-Simulation Adjustments

- Adjusted clock divider settings to achieve optimal scrolling speeds and animation fluidity.
- Debugged any issues related to memory management and switching logic.

The simulation in **Quartus Altera** was crucial in refining the design, ensuring a reliable and efficient implementation on hardware. By combining this rigorous design method with robust simulation, the project successfully achieved its intended objectives.

4 Implementation

4.1 Hardware Setup

The CPLD development board is connected to the 8x8 LED matrix display through row and column lines. Switches SW1 to SW8 are connected to input pins on the CPLD to allow user interaction. A stable clock source is provided to the CPLD to ensure synchronous operation of the display logic.

Table: Switch assignment

Switch (pin 47-63)	Stored Data
Sw1 (pin 47)	Display- 'ENTRY'
Sw2 (pin 48)	Display- 'EXIT'
Sw3 (pin 49)	Command for Left Scrolling Content
Sw4 (pin 51)	Command for Right Scrolling Content
Sw5 (pin 59)	Command for Up Scrolling Content

Sw6 (pin 60)	Command for Down Scrolling Content
Sw7 (pin 62)	Display- 'STOP'
Sw8 (pin 63)	Animation of a Walking Person

4.2 Software Development

The Verilog code is structured to initialize display patterns, manage clock signals, and handle user inputs. The implementation involves:

- **Initialization:** Preloading the display memory with binary patterns representing characters like "OPEN", "EXIT", and symbols such as arrows and a walking person.
- **Clock Division:** Creating slower clock signals from the system clock to control scrolling and animation timing and frequency.
- **State Management:** Using counters and state variables to keep track of the current display position and active display mode.
- **Display Control:** Continuously updating the row and column outputs to reflect the current state of the display memory based on the active mode and user inputs.

Functional Snippets

```
display_col_memory [0] <= 8'b10000000;
display_col_memory [1] <= 8'b01000000;
display_col_memory [2] <= 8'b00100000;
display_col_memory [3] <= 8'b00010000;
display_col_memory [4] <= 8'b00001000;
display_col_memory [5] <= 8'b00000100;
display_col_memory [6] <= 8'b00000010;
display_col_memory [7] <= 8'b00000001;
```

Fixed column designation

```
//ENTRY/SW1
display_row_memory[8] <= 8'b00000000;//E
display_row_memory[9] <= 8'b11111111;
display_row_memory[10] <= 8'b11111111;
display_row_memory[11] <= 8'b11011011;
display_row_memory[12] <= 8'b11011011;
display_row_memory[13] <= 8'b11011011;
display_row_memory[14] <= 8'b11011011;
display_row_memory[15] <= 8'b00000000;
```

Binary data for individual character

```
//col clock
always @(posedge clk) begin
    count1 <= count1+1;
    if (count1 >=10000) begin
        count1 <=0;
        clkd = ~clkd;
    end
end
//scroll clock
always @(posedge clk) begin
    count2 <= count2+1;
    if (count2 >=2000000) begin
        count2 <=0;
        clk_scroll = ~clk_scroll;
    end
end
```

Separate frequency controlling of clocks

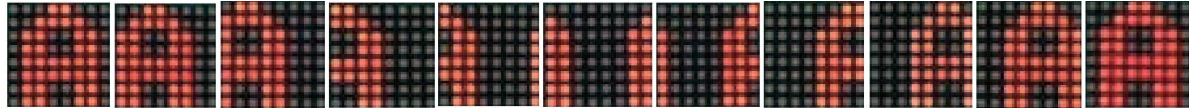
```
//frame clock
always @(posedge clk) begin
    count3 <= count3+1;
    if (count3 >=1000000) begin
        count3 <=0;
        clk_frame = ~clk_frame;
    end
end
```

4.3 Testing

The system was tested by cycling through different switch inputs to verify the correct display of messages and animations. Scrolling text was observed for smooth movement, and animations were checked for proper frame sequencing. Reset functionality was also tested to ensure the system returns to the default state when triggered.

4.4 Results

The test confirms the system functions as intended, with correct message and animation displays, smooth scrolling text, proper animation sequencing, and reliable reset functionality. These results indicate the system is robust and ready for use.



4.5 Github Link

For more details, refer to the project's GitHub repository: <https://github.com/kay33d/CPLD-digital-sinage> .

4.6 Google Drive Link

For a detailed walkthrough, check out the video on : https://drive.google.com/drive/folders/1DNTFCHYRaVswgt7Q_mbn5U4-l31f4bF7 .

5 Design Analysis and Evaluation

5.1 Novelty

- **CPLD-Based Design:** Offers dynamic reconfiguration and efficient resource usage compared to traditional microcontroller-based systems.
- **Interactive Features:** Supports customizable display modes and animations through user input.
- **Modular Verilog Architecture:** Ensures scalability and resource optimization.
- **Cost-Effective and Scalable:** Suitable for a wide range of applications, from small-scale to larger installations.
- **Innovative Techniques:** Implements smooth animations and seamless text scrolling using programmable logic.

5.2 Performance

The digital signage system successfully displays predefined messages and animations. Scrolling text operates smoothly at the intended speed, and animations transition seamlessly between frames. The responsiveness to switch inputs is prompt, allowing users to change display modes without noticeable delay.

5.3 Resource Utilization

The CPLD efficiently handles the required logic without overutilization of resources. The design leverages the programmable nature of the CPLD to manage multiple functionalities within a single module, optimizing both space and power consumption.

5.4 Reliability

During testing, the system demonstrated consistent performance without errors or glitches in the display output. The reset functionality effectively restores the system to a known state, ensuring reliability in prolonged operations.

5.5 Limitations

- **Display Resolution:** The 8x8 matrix limits the complexity and size of displayable content.
- **Scalability:** Extending the display to larger matrices would require modifications to the design and potentially more complex control logic.
- **User Interface:** The current switch-based input is basic and could be enhanced with more advanced interfaces for greater control.

5.6 Results and Analysis

Since this project involves a dynamic display system where characters can be shown while scrolling on the 8x8 matrix, we are incorporating images of selected animation frames into the report. These frames illustrate the smooth transitions and motion effects achieved through the animation module, providing a clear visual representation of the system's capabilities.

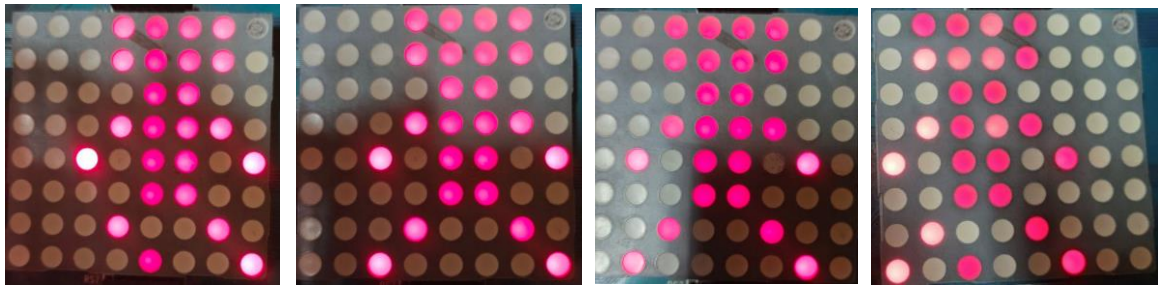


Figure: Frames from Animation of a 'Walking Person' (left to right scrolling)

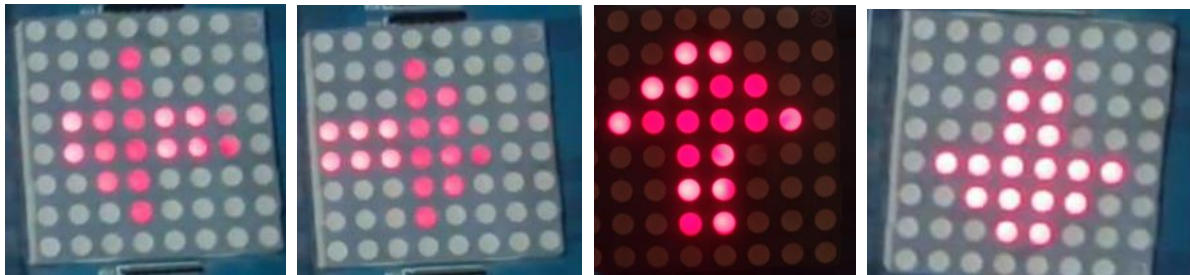


Figure: Glimpse from left, right, up and down scrolling

6 Reflection on Individual and Team work (PO(i))

6.1 Individual Contributions

Each team member contributed to different aspects of the project, including hardware setup, Verilog coding, testing, and documentation. Personal strengths were leveraged to ensure a balanced workload and efficient progress.

6.2 Team Dynamics

The team maintained effective communication through regular meetings and collaborative tools. Challenges such as debugging the Verilog code and coordinating hardware connections were addressed collectively, fostering a supportive and productive environment.

6.3 Lessons Learned

- **Importance of Modular Design:** Breaking down the system into manageable modules facilitated easier debugging and future enhancements.
- **Effective Communication:** Clear and consistent communication was crucial in aligning team efforts and resolving issues promptly.
- **Time Management:** Adhering to a project timeline ensured timely completion of milestones and overall project success.

6.4 Log Book of Project Implementation

Date	Milestone Achieved	Individual Role	Team Role	Comments
09/10/2024	Project Approved	-	-	-
10/10/2024	Project Planning	Kaidul, Sadaf, Rifat, Fabliha	Collaborated to outline project workflow	Established initial roles and tasks
10/10/2024	Literature Review	Kaidul, Sadaf, Rifat	Shared insights from related research	Reviewed CPLD-based LED display systems
14/10/2024	Hardware Selection	Fabliha, Rifat	Discussed and finalized hardware list	Selected CPLD and 8x8 LED matrix
14/10/2024	Problem Formulation	Rifat, Kaidul	Refined scope and problem statement	Defined key challenges and constraints
21/10/2024	Verilog Code Drafting	Rifat, Fabliha	Collaboratively debugged and reviewed code	Wrote initial Verilog modules for display control
21/10/2024	Simulation Setup	Rifat, Sadaf	Tested modules for timing and synchronization	Configured Quartus testbench for simulation
28/10/2024	Hardware Integration	Sadaf, Kaidul, Fabliha	Verified stability of hardware connections	Connected CPLD to the 8x8 LED matrix
28/10/2024	Clock Signal Configuration	Rifat, Kaidul	Aligned clock speeds with animation timing	Designed and tested clock divider modules
04/11/2024	Presentation on Project Progress	Kaidul, Sadaf, Rifat, Fabliha	Demonstrated initial progress.	Valuable insights from course instructors on implementation.

11/11/2024	Simulation Refinement	Rifat, Fabliha, Kaidul	Tested multiple display modes and scenarios	Enhanced Verilog code for advanced animations
18/11/2024	Switch Functionality Testing	Sadaf, Kaidul, Rifat	Verified mapping of switches to modes	Validated user inputs via switches
18/11/2024	Functional Testing	Rifat, Kaidul, Sadaf	Collected feedback for improvements	Conducted testing of all features
25/11/2024	Debugging and Optimization	Rifat, Fabliha	Reviewed overall system performance	Identified and fixed coding and hardware issues
04/12/2024	Documentation	Sadaf, Fabliha	Combined and edited individual contributions	Wrote technical sections of the project report
04/12/2024	User Manual Drafting	Sadaf, Fabliha	Peer-reviewed and finalized the manual	Created instructions for operating the display
08/12/2024	Presentation Preparation	Kaidul, Fabliha, Sadaf	Practiced and refined the presentation	Designed presentation slides and talking points
08/12/2024	Final Review	Rifat, Kaidul	Confirmed readiness for final submission	Reviewed the project report and presentation

7 Executive Summary

7.1 Executive Summary

The digital signage project successfully developed a versatile display system using an 8x8 dot matrix LED controlled by an LP-2900-CPLD development board. The system supports scrolling text, static messages, and simple animations, all managed through user-controlled switches. The implementation demonstrates the effective use of programmable logic in creating dynamic visual displays suitable for various applications. The project was completed within the allocated time and budget, highlighting efficient project management and team collaboration. Future enhancements could expand display capabilities and improve user interaction interfaces, further increasing the system's applicability and functionality.

7.2 User Manual

7.2.1 Introduction

This user manual provides instructions for operating the digital signage system developed using the LP-2900-CPLD development board and an 8x8 dot matrix LED display.

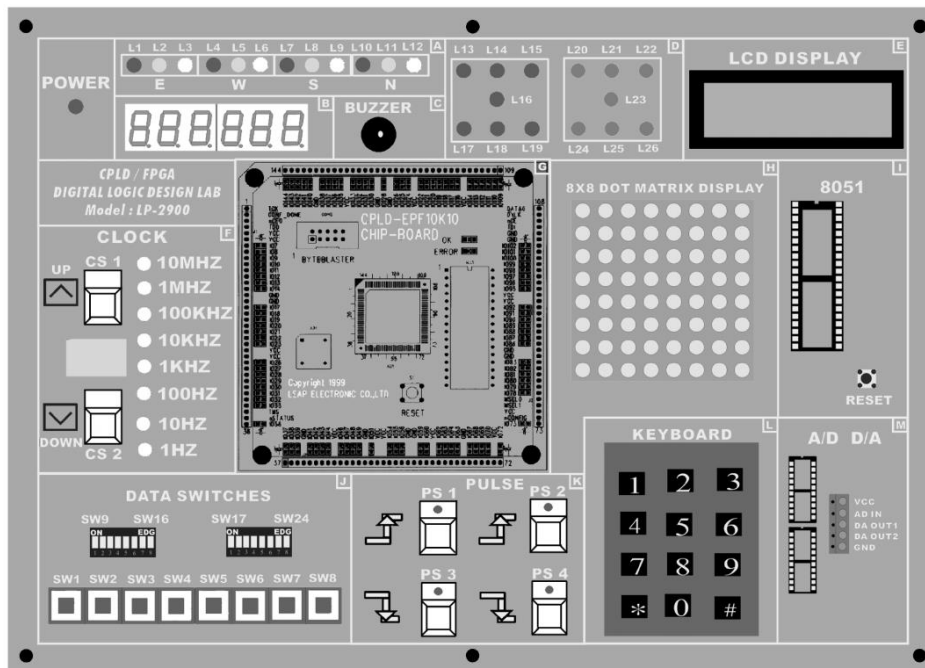


Figure: LP-2900 Logic Design Lab Platform

7.2.2 System Components

- **LED Matrix Display:** The 8x8 grid that shows messages and animations.
- **Switches (SW1 to SW8):** Controls to change display modes and animations.
- **Power Supply:** Ensures the system is powered on.

7.2.3 Operating Instructions

1. **Powering On:** Connect the power supply to the CPLD development board and turn it on.
2. **Default Display:** Upon startup, the display will show the default message "OPEN".

3. **Changing Display Modes:**
 - **SW1 (Entry/SW1):** Displays the "ENTRY" message.
 - **SW2 (Exit/SW2):** Displays the "EXIT" message.
 - **SW3 (Right Arrow/Down SW6):** Shows a right-pointing arrow or downward animation.
 - **SW4 (Left Arrow/Up SW5):** Shows a left-pointing arrow or upward animation.
 - **SW7 (Stop/SW7):** Displays the "STOP" message.
 - **SW8 (Walking Person/SW8):** Initiates the walking person animation.
4. **Scrolling Text:** Use the appropriate switches to activate scrolling text. The text will move horizontally across the display.
5. **Resetting the Display:** Press the reset button to return the display to the default "OPEN" message.

7.2.4 Troubleshooting

- **No Display Output:** Ensure the power supply is connected and the CPLD is powered on.
- **Unresponsive Switches:** Check the connections of the switches to the CPLD and verify switch functionality.
- **Flickering Display:** Verify the stability of the clock signal and ensure all connections are secure.

8 Project Management and Cost Analysis (PO(k))

8.1 Project Management

The project followed a structured approach divided into phases:

1. **Planning:** Defined project objectives, scope, and timelines.
2. **Design:** Developed hardware schematics and Verilog code structure.
3. **Implementation:** Assembled hardware components and wrote/verilog code.
4. **Testing:** Conducted functionality tests and debugging.
5. **Documentation:** Compiled the report, user manual, and cost analysis.
6. **Review and Evaluation:** Assessed project outcomes and identified areas for improvement.

Regular team meetings and progress tracking ensured adherence to the project timeline. Tools such as version control systems and collaborative platforms were utilized to facilitate teamwork.

8.2 Cost Analysis

As this project utilized the LP-2900-CPLD development board provided by the lab and Quartus Altera software, which is free to use, no additional costs were incurred for hardware or software. This setup highlights the affordability of using available lab resources for educational and developmental purposes.

8.3 Budget Justification

- **CPLD Development Board:** Essential for programmable logic operations.
- **LED Matrix Display:** Core component for visual output.
- **Switches:** Required for user interaction and control.
- **Power Supply:** Necessary for system operation.

9 Future Work (PO(I))

9.1 Enhancements

- **Larger Display Matrix:** Expanding to a higher resolution LED matrix for more complex and detailed displays.
- **Wireless Connectivity:** Implementing wireless modules to allow remote updates and control of display content.
- **Advanced User Interface:** Introducing touch controls or mobile app integration for more intuitive user interactions.
- **Dynamic Content Loading:** Enabling the system to load content from external sources, such as USB drives or network streams.

9.2 Additional Features

- **Real-Time Clock Integration:** Displaying time-based messages or schedules.
- **Sensor Integration:** Incorporating sensors to change displays based on environmental inputs (e.g., motion sensors).
- **Energy Efficiency:** Optimizing power consumption for prolonged usage and sustainability.
- **Interactive Animations:** Developing more sophisticated animations and interactive patterns to enhance user engagement.

9.3 Scalability

Future iterations of the project can explore scalability by networking multiple signage units, enabling synchronized displays across different locations. This would be beneficial for large-scale advertising campaigns or informational displays in expansive venues.

10 References

- [1] Z. Yuan-yuan and Z. Hai-dong, "Design and Implementation of LED Dot-Matrix Display System Based on FPGA," *Proc. 2011 Int. Conf. Electron., Commun. and Control (ICECC)*, Ningbo, China, 2011,.
- [2] Z. Hu and Y. Zhang, "Design of Large-Scale LED Display System Based on FPGA," *Int. J. Adv. Comput. Technol.*, Dec. 2012.
- [3] H. Kaur and P. Singh, "FPGA-Based Display Driver Design for High-Speed LED Display," *Int. J. Eng. Adv. Technol. (IJEAT)*, June 2019.
- [4] W. Wolf, *FPGA-Based System Design*, 1st ed., Upper Saddle River, NJ, USA: Prentice Hall, 2004.

11 Appendix

Code

```
module Open(
    input clk,          // System clock, clk = 1k
    input reset,        // Reset signal assign pulse
    input [7:0]switch,  // Switch
    output reg [7:0] row, // Rows of LED matrix
    output reg [7:0] col // Columns of LED matrix
);

// Declare state variables
reg [7:0] display_row_memory [0:500]; // Memory for storing display data (text: "OPEN")
reg [7:0] display_col_memory [0:7];
reg [8:0] scroll_counter; // Counter for scrolling text
reg clk_d;
reg [23:0] count1;
reg clk_scroll;
reg [23:0] count2;
reg [2:0] col_counter;
reg [8:0] scroll_counter_initial;
reg [8:0] scroll_counter_final;

reg clk_frame;
reg [23:0] count3;
reg [5:0] frame_counter;
// Initialize the display memory with "OPEN" text pattern (binary representation of each character)
initial begin
    // Binary data for scrolling "OPEN" on 8x8 matrix
    col_counter <= 0;
    scroll_counter <= 0;
    count1 <= 0;
    clk_d <= 0;
    clk_scroll <= 0;
    count2 <= 0;
    scroll_counter_initial <= 0;
    scroll_counter_final <= 0;

    display_col_memory [0] <= 8'b10000000;
    display_col_memory [1] <= 8'b01000000;
    display_col_memory [2] <= 8'b00100000;
    display_col_memory [3] <= 8'b00010000;
    display_col_memory [4] <= 8'b00001000;
    display_col_memory [5] <= 8'b00000100;
    display_col_memory [6] <= 8'b00000010;
    display_col_memory [7] <= 8'b00000001;

    //Blank
    display_row_memory[0] <= 8'b00000000; //blank frame
    display_row_memory[1] <= 8'b00000000;
    display_row_memory[2] <= 8'b00000000;
    display_row_memory[3] <= 8'b00000000;
    display_row_memory[4] <= 8'b00000000;
    display_row_memory[5] <= 8'b00000000;
    display_row_memory[6] <= 8'b00000000;
    display_row_memory[7] <= 8'b00000000;

    //ENTRY/SW1
    display_row_memory[8] <= 8'b00000000; //E
    display_row_memory[9] <= 8'b11111111;
    display_row_memory[10] <= 8'b11111111;
    display_row_memory[11] <= 8'b11011011;
    display_row_memory[12] <= 8'b11011011;
    display_row_memory[13] <= 8'b11011011;
    display_row_memory[14] <= 8'b11011011;
    display_row_memory[15] <= 8'b00000000;

    display_row_memory[16] <= 8'b00000000; //N
    display_row_memory[17] <= 8'b11111111;
    display_row_memory[18] <= 8'b01111111;
    display_row_memory[19] <= 8'b00111000;
    display_row_memory[20] <= 8'b00011100;
    display_row_memory[21] <= 8'b11111110;
    display_row_memory[22] <= 8'b11111111;
```

```

display_row_memory[23] <= 8'b00000000;

display_row_memory[24] <= 8'b00000000;//T
display_row_memory[25] <= 8'b11000000;
display_row_memory[26] <= 8'b11000000;
display_row_memory[27] <= 8'b11111111;
display_row_memory[28] <= 8'b11111111;
display_row_memory[29] <= 8'b11000000;
display_row_memory[30] <= 8'b11000000;
display_row_memory[31] <= 8'b00000000;

display_row_memory[32] <= 8'b00000000;//R
display_row_memory[33] <= 8'b11111111;
display_row_memory[34] <= 8'b11111111;
display_row_memory[35] <= 8'b11011100;
display_row_memory[36] <= 8'b11011110;
display_row_memory[37] <= 8'b11111011;
display_row_memory[38] <= 8'b01110001;
display_row_memory[39] <= 8'b00000000;

display_row_memory[40] <= 8'b00000000;//Y
display_row_memory[41] <= 8'b11000000;
display_row_memory[42] <= 8'b01100000;
display_row_memory[43] <= 8'b00111111;
display_row_memory[44] <= 8'b00111111;
display_row_memory[45] <= 8'b01100000;
display_row_memory[46] <= 8'b11000000;
display_row_memory[47] <= 8'b00000000;
//EXIT/sw2
display_row_memory[48] <= 8'b00000000;//E
display_row_memory[49] <= 8'b11111111;
display_row_memory[50] <= 8'b11111111;
display_row_memory[51] <= 8'b11011011;
display_row_memory[52] <= 8'b11011011;
display_row_memory[53] <= 8'b11011011;
display_row_memory[54] <= 8'b11011011;
display_row_memory[55] <= 8'b00000000;

display_row_memory[56] <= 8'b00000000; //X
display_row_memory[57] <= 8'b10000001;
display_row_memory[58] <= 8'b11000011;
display_row_memory[59] <= 8'b01100110;
display_row_memory[60] <= 8'b00111100;
display_row_memory[61] <= 8'b01100110;
display_row_memory[62] <= 8'b11000011;
display_row_memory[63] <= 8'b10000001;

display_row_memory[64] <= 8'b00000000;//I
display_row_memory[65] <= 8'b10000001;
display_row_memory[66] <= 8'b11111111;
display_row_memory[67] <= 8'b11111111;
display_row_memory[68] <= 8'b10000001;

display_row_memory[69] <= 8'b00000000;//T
display_row_memory[70] <= 8'b11000000;
display_row_memory[71] <= 8'b11000000;
display_row_memory[72] <= 8'b11111111;
display_row_memory[73] <= 8'b11111111;
display_row_memory[74] <= 8'b11000000;
display_row_memory[75] <= 8'b11000000;
display_row_memory[76] <= 8'b00000000;
display_row_memory[77] <= 8'b00000000;
//right arrow/sw3/down sw6
display_row_memory[78] <= 8'b00000000;
display_row_memory[79] <= 8'b00011000;
display_row_memory[80] <= 8'b00011000;
display_row_memory[81] <= 8'b00011000;
display_row_memory[82] <= 8'b01111110;
display_row_memory[83] <= 8'b00111100;
display_row_memory[84] <= 8'b00011000;
display_row_memory[85] <= 8'b00000000;
//left arrow/sw4/up sw5
display_row_memory[86] <= 8'b00000000;
display_row_memory[87] <= 8'b00011000;
display_row_memory[88] <= 8'b00111100;
display_row_memory[89] <= 8'b01111110;

```

```

display_row_memory[90] <= 8'b00011000;
display_row_memory[91] <= 8'b00011000;
display_row_memory[92] <= 8'b00011000;
display_row_memory[93] <= 8'b00000000;
//STOP/sw7
display_row_memory[94] <= 8'b00000000;//S
display_row_memory[95] <= 8'b01110011;
display_row_memory[96] <= 8'b11111011;
display_row_memory[97] <= 8'b11011011;
display_row_memory[98] <= 8'b11011011;
display_row_memory[99] <= 8'b11011111;
display_row_memory[100] <= 8'b11001110;
display_row_memory[101] <= 8'b00000000;

display_row_memory[102] <= 8'b00000000;//T
display_row_memory[103] <= 8'b11000000;
display_row_memory[104] <= 8'b11000000;
display_row_memory[105] <= 8'b11111111;
display_row_memory[106] <= 8'b11111111;
display_row_memory[107] <= 8'b11000000;
display_row_memory[108] <= 8'b11000000;
display_row_memory[109] <= 8'b00000000;

display_row_memory[110] <= 8'b00000000;//O
display_row_memory[111] <= 8'b00111100;
display_row_memory[112] <= 8'b01111110;
display_row_memory[113] <= 8'b10000001;
display_row_memory[114] <= 8'b10000001;
display_row_memory[115] <= 8'b01111110;
display_row_memory[116] <= 8'b00111100;
display_row_memory[117] <= 8'b00000000;

display_row_memory[118] <= 8'b00000000;//p
display_row_memory[119] <= 8'b11111111;
display_row_memory[120] <= 8'b11111111;
display_row_memory[121] <= 8'b11011000;
display_row_memory[122] <= 8'b11011000;
display_row_memory[123] <= 8'b11111000;
display_row_memory[124] <= 8'b01110000;
display_row_memory[125] <= 8'b00000000;

////walking person/sw8
display_row_memory[126] <= 8'b00000000;//frame1
display_row_memory[127] <= 8'b00001001;
display_row_memory[128] <= 8'b11010010;
display_row_memory[129] <= 8'b11111100;
display_row_memory[130] <= 8'b11111100;
display_row_memory[131] <= 8'b11010010;
display_row_memory[132] <= 8'b00001001;
display_row_memory[133] <= 8'b00000000;

display_row_memory[134] <= 8'b00000000;//frame2
display_row_memory[135] <= 8'b00001000;
display_row_memory[136] <= 8'b11010010;
display_row_memory[137] <= 8'b11111101;
display_row_memory[138] <= 8'b11111100;
display_row_memory[139] <= 8'b11010010;
display_row_memory[140] <= 8'b00001001;
display_row_memory[141] <= 8'b00000000;

display_row_memory[142] <= 8'b00000000;//frame3
display_row_memory[143] <= 8'b00001001;
display_row_memory[144] <= 8'b11010010;
display_row_memory[145] <= 8'b11111100;
display_row_memory[146] <= 8'b11111100;
display_row_memory[147] <= 8'b11010010;
display_row_memory[148] <= 8'b00001001;
display_row_memory[149] <= 8'b00000000;

display_row_memory[150] <= 8'b00000000;//frame3
display_row_memory[151] <= 8'b00001001;
display_row_memory[152] <= 8'b11010010;
display_row_memory[153] <= 8'b11111100;
display_row_memory[154] <= 8'b11111101;
display_row_memory[155] <= 8'b11010010;
display_row_memory[156] <= 8'b00001000;

```

```

        display_row_memory[157] <= 8'b00000000;

//error
        display_row_memory[158] <= 8'b00111100;
        display_row_memory[159] <= 8'b01000010;
        display_row_memory[160] <= 8'b10100101;
        display_row_memory[161] <= 8'b10011001;
        display_row_memory[162] <= 8'b10011001;
        display_row_memory[163] <= 8'b10100101;
        display_row_memory[164] <= 8'b01000010;
        display_row_memory[165] <= 8'b00111100;

end
//col clock
always @(posedge clk) begin
    count1 <= count1+1;
    if (count1 >=10000) begin
        count1 <=0;
        clk_d = ~clk_d;
    end
end
//scroll clock
always @(posedge clk) begin
    count2 <= count2+1;
    if (count2 >=2000000) begin
        count2 <=0;
        clk_scroll = ~clk_scroll;
    end
end
//frame clock
always @(posedge clk) begin
    count3 <= count3+1;
    if (count3 >=1000000) begin
        count3 <=0;
        clk_frame = ~clk_frame;
    end
end

//col control logic
always @(posedge clk_d or posedge reset) begin
    if (reset) begin
        col_counter <= 0;
    end
    else begin
        col_counter <= col_counter + 1;
        if (col_counter >= 7) // Loop the text
            col_counter <= 0;
    end
end

//frame control logic
always @(posedge clk_frame or posedge reset) begin
    if (reset) begin
        frame_counter <= 0;
    end
    else if (switch==8'b00000001) begin
        frame_counter <= frame_counter + 8;
        if (frame_counter >= 24) // Loop the frame
            frame_counter <= 0;
    end
    else begin
        frame_counter <= 0;
    end
end

//// Scrolling control logic
always @(*) begin
    case(switch)
        8'b00000000: begin
            scroll_counter_initial <=0;
            scroll_counter_final <=7;

            end

        8'b10000000: begin
            scroll_counter_initial <=8;
            scroll_counter_final <=47;

            end
    endcase
end

```

```

8'b01000000: begin
    scroll_counter_initial <=48;
    scroll_counter_final <=77;
end

8'b00100000: begin
    scroll_counter_initial =78;
    scroll_counter_final =85;
end

8'b00010000: begin
    scroll_counter_initial =86;
    scroll_counter_final =93;
end

8'b00001000: begin
    scroll_counter_initial =86;
    scroll_counter_final =93;
end

8'b00000100: begin
    scroll_counter_initial =78;
    scroll_counter_final =85;
end

8'b00000010: begin
    scroll_counter_initial =94;
    scroll_counter_final =125;
end

8'b00000001: begin
    scroll_counter_initial =126;
    scroll_counter_final =133;
end

default : begin//multiple switch
    scroll_counter_initial =158;
    scroll_counter_final =158;
end

endcase
end

always @(posedge clk_scroll or posedge reset) begin
    if (reset) begin
        scroll_counter <= scroll_counter_initial;
    end
    else if((switch==8'b00100000)||((switch==8'b00000100))) begin
        scroll_counter <= scroll_counter - 1;
        if (scroll_counter <= scroll_counter_initial)
            scroll_counter <= scroll_counter_final;
        if (scroll_counter > scroll_counter_final)
            scroll_counter <= scroll_counter_final;
    end
    else begin
        if (scroll_counter < scroll_counter_initial)
            scroll_counter <= scroll_counter_initial;
        else if (scroll_counter >= scroll_counter_final)
            scroll_counter <= scroll_counter_initial;
        else scroll_counter <= scroll_counter + 1;
    end
end

// LED Matrix control logic
always @(posedge clk_d) begin
    // Set row and column data based on scrolling counter (fade effect)
    if((switch==8'b00001000)||((switch==8'b00000100))) begin
        row <= display_col_memory[col_counter]; // Get column data for current scroll position
        if ((col_counter+scroll_counter)>scroll_counter_final)
            col <= display_row_memory[scroll_counter_initial+(col_counter+scroll_counter)-
scroll_counter_final];
        else
            col <= display_row_memory[col_counter+scroll_counter];
    end
    else begin
        col <= display_col_memory[col_counter]; // Get column data for current scroll position
        if ((col_counter+scroll_counter+frame_counter)>scroll_counter_final)
            row <= display_row_memory[scroll_counter_initial+(col_counter+scroll_counter+frame_counter)-scroll_counter_final];
        else
            row <= display_row_memory[col_counter+scroll_counter+frame_counter];
    end
end
end
endmodule

```